

树上算法

Morning Flower and Evening Oath

XCPC vector 模板分册

目录

使用约定	ii
第一章 树上算法	1
1.1 重链剖分与 LCA	1
第二章 离线最近公共祖先	3
2.1 离线 Tarjan LCA	3
第三章 在线最近公共祖先	5
3.1 欧拉序与 ST 表 LCA	5
第四章 树上点集摘要	8
4.1 树上点集直径摘要	8
第五章 倍增祖先与路径最值	10
5.1 倍增 LCA、祖先与最大边权	10
第六章 树上路径交	13
6.1 两条树路径的交集	13
第七章 点分治	14
7.1 点分治：树上距离点对计数	14
第八章 树上启发式合并	17
8.1 树上启发式合并：子树众数颜色和	17
第九章 虚树	19
9.1 虚树：关键点压缩与路径信息	19
第十章 树上路径第 k 小	22
10.1 树上路径点权第 k 小	22
第十一章 动态树	25
11.1 Link-cut tree：动态森林与路径异或	25
第十二章 配套知识与验证范围	28
12.1 非负权树的点集直径摘要	28

使用约定

C++20，默认包含 `bits/stdc++.h` 并使用 `std` 命名空间。下标、区间及数值范围以各条目说明为准。DFS 保持递归，需满足题目栈空间条件。

本册按“大类（分册）—种类（章）—项目（节）”组织。跨类引用的已收录组件附在依赖部分；源头文件中的依赖仍须一并检查。

本册尚非全量完成稿；传统版评测仅为历史档案，当前只维护 `vector` 代码。

第一章 树上算法

1.1 重链剖分与 LCA

add(u,v) 建无向树; build(root) 后读取 fa、dep、siz、son、top、dfn、rk。子树对应 [dfn[u],dfn[u]+siz[u]-1]。lca(u,v) 为 $O(\log n)$, path(u,v,work,edge) 分解路径; work(l,r) 仅适用于可交换聚合, 边权模式排除 LCA 位置。输入必须是连通无环树, 建树 $O(n)$, 两遍递归 DFS; 支持改变根后重复 build, dfn 会重新编号。

```
1 struct HLD
2 {
3     int n, timer = 0;
4     vector<vector<int>> g;
5     vector<int> fa, dep, siz, son, top, dfn, rk;
6
7     HLD(int n)
8         : n(n),
9           g(n + 1),
10          fa(n + 1),
11          dep(n + 1),
12          siz(n + 1),
13          son(n + 1),
14          top(n + 1),
15          dfn(n + 1),
16          rk(n + 1)
17     {
18     }
19
20     void add(int u, int v)
21     {
22         g[u].push_back(v);
23         g[v].push_back(u);
24     }
25
26     void dfs1(int u, int p)
27     {
28         fa[u] = p;
29         siz[u] = 1;
30         son[u] = 0;
31         for (int v : g[u])
32         {
33             if (v == p) continue;
34             dep[v] = dep[u] + 1;
35             dfs1(v, u);
36             siz[u] += siz[v];
37             if (!son[u] || siz[v] > siz[son[u]]) son[u] = v;
38         }
39     }
40
41     void dfs2(int u, int t)
42     {
43         top[u] = t;
44         dfn[u] = ++timer;
```

```

45     rk[timer] = u;
46     if (son[u]) dfs2(son[u], t);
47     for (int v : g[u])
48     {
49         if (v != fa[u] && v != son[u]) dfs2(v, v);
50     }
51 }
52
53 // Connected tree only. Recursive DFS; subtree interval is [dfn, dfn+siz-1].
54 void build(int root = 1)
55 {
56     assert(1 <= root && root <= n);
57     timer = 0;
58     dep[root] = 0;
59     dfs1(root, root);
60     dfs2(root, root);
61     assert(timer == n);
62 }
63
64 int lca(int u, int v) const
65 {
66     while (top[u] != top[v])
67     {
68         if (dep[top[u]] < dep[top[v]]) swap(u, v);
69         u = fa[top[u]];
70     }
71     return dep[u] < dep[v] ? u : v;
72 }
73
74 // Commutative operations only; edge=true excludes the LCA's position.
75 template <class F> void path(int u, int v, F work, bool edge = false) const
76 {
77     while (top[u] != top[v])
78     {
79         if (dep[top[u]] < dep[top[v]]) swap(u, v);
80         work(dfn[top[u]], dfn[u]);
81         u = fa[top[u]];
82     }
83     if (dep[u] > dep[v]) swap(u, v);
84     if (dfn[u] + edge <= dfn[v]) work(dfn[u] + edge, dfn[v]);
85 }
86 };

```

第二章 离线最近公共祖先

2.1 离线 Tarjan LCA

构造 `OfflineLCA(n)`, `add(u,v)` 加无向树边, `add_query(u,v)` 登记查询并返回从 0 开始的编号, `run(root=1)` 后读取 `answer[id]`。输入必须是非空树, 点号 $1..n$; 允许同点、重复查询及零个查询。`run` 可重复调用或换根, 不清空查询; 追加查询后需重新 `run`, 旧答案在重算前不可按新根使用。DFS 与并查集查找均递归。并查集按大小合并并压缩路径, `ancestor[find(u)]` 保存已合并部分对应的当前树祖先; 不能直接拿并查集代表当 LCA。每个子树完成后合并到父点并更新 `ancestor`; 查询另一端已完成时, 从其集合读取祖先。总时间 $O((n+q) \alpha(n))$, 空间 $O(n+q)$ 。全部点对和根的父链对拍、50 万点链及 P3379、POJ1470 完整驱动本地验证, 在线待补。

```
1 struct OfflineLCA
2 {
3     int n;
4     vector<vector<int>>> g;
5     vector<vector<pair<int, int>>>> queries;
6     vector<int> parent, size, ancestor, done, answer;
7
8     OfflineLCA(int n)
9         : n(n),
10          g(n + 1),
11          queries(n + 1),
12          parent(n + 1),
13          size(n + 1),
14          ancestor(n + 1),
15          done(n + 1)
16     {
17     }
18
19     void add(int u, int v)
20     {
21         assert(1 <= u && u <= n && 1 <= v && v <= n);
22         g[u].push_back(v);
23         g[v].push_back(u);
24     }
25
26     int add_query(int u, int v)
27     {
28         assert(1 <= u && u <= n && 1 <= v && v <= n);
29         int id = answer.size();
30         answer.push_back(0);
31         queries[u].push_back({v, id});
32         queries[v].push_back({u, id});
33         return id;
34     }
```

```

36     int find(int u)
37     {
38         if (parent[u] != u) parent[u] = find(parent[u]);
39         return parent[u];
40     }
41
42     void merge(int u, int v)
43     {
44         u = find(u);
45         v = find(v);
46         if (size[u] < size[v]) swap(u, v);
47         parent[v] = u;
48         size[u] += size[v];
49     }
50
51     void dfs(int u, int p)
52     {
53         for (int v : g[u])
54             if (v != p)
55             {
56                 dfs(v, u);
57                 merge(u, v);
58                 ancestor[find(u)] = u;
59             }
60         done[u] = 1;
61         for (auto [v, id] : queries[u])
62             if (done[v]) answer[id] = ancestor[find(v)];
63     }
64
65     // Input must be a nonempty tree. Query IDs are 0-based.
66     void run(int root = 1)
67     {
68         assert(1 <= root && root <= n);
69         iota(parent.begin(), parent.end(), 0);
70         iota(ancestor.begin(), ancestor.end(), 0);
71         fill(size.begin(), size.end(), 1);
72         fill(done.begin(), done.end(), 0);
73         fill(answer.begin(), answer.end(), 0);
74         dfs(root, 0);
75     }
76 };

```

第三章 在线最近公共祖先

3.1 欧拉序与 ST 表 LCA

构造 EulerLCA(n), add(u,v,w=1) 加非负权无向树边, build(root=1) 预处理; lca(u,v) 返回最近公共祖先, distance(u,v) 仍返回无权路径边数; weighted_distance(u,v) 返回 long long 带权距离。边权非负, 所有树上路径的权和须 $\leq \text{LLONG_MAX}$ 。输入必须为非空树, 点号 1..n; 允许同点查询和重复 build 换根。只可在 build 后查询, 加边会清除 ready 标志。递归 DFS 在进入顶点及从每个儿子返回时记录顶点, euler 长度为 $2n-1$, first[u] 是从 0 开始的首次出现位置; depth[root]=0。两点首次出现位置间深度最小的顶点就是 LCA, 用 ST 表保存该顶点。预处理 $O(n \log n)$ 时间和空间, 每次查询 $O(1)$ 。ST 内部下标从 0 开始; first、depth、euler、root_dist 可直接读取; root_dist 为当前根的带权距离。50 万点链、全部小树根与点对以及 P3379、POJ1330 双风格完整程序本地验证; 上述无权题在线待补; QOJ8235 的带权应用已双 AC。

```
1 struct EulerLCA
2 {
3     using ll = long long;
4     int n;
5     bool ready = false;
6     vector<vector<int>> g, st;
7     vector<vector<ll>> cost;
8     vector<ll> root_dist;
9     vector<int> first, depth, euler, lg;
10
11     EulerLCA(int n)
12         : n(n), g(n + 1), cost(n + 1), root_dist(n + 1), first(n + 1), depth(n + 1)
13     {
14     }
15
16     void add(int u, int v, ll w = 1)
17     {
18         assert(1 <= u && u <= n && 1 <= v && v <= n);
19         assert(w >= 0);
20         g[u].push_back(v);
21         g[v].push_back(u);
22         cost[u].push_back(w);
23         cost[v].push_back(w);
24         ready = false;
25     }
26
27     int shallower(int u, int v) const { return depth[u] <= depth[v] ? u : v; }
```

递归欧拉序与 RMQ (接上页同一结构体):

```

29 void dfs(int u, int parent)
30 {
31     first[u] = euler.size();
32     euler.push_back(u);
33     for (int i = 0; i < int(g[u].size()); i++)
34     {
35         int v = g[u][i];
36         if (v != parent)
37         {
38             depth[v] = depth[u] + 1;
39             root_dist[v] = root_dist[u] + cost[u][i];
40             dfs(v, u);
41             euler.push_back(u);
42         }
43     }
44 }
45
46 // Input must be a nonempty tree. Rebuilding may change the root.
47 void build(int root = 1)
48 {
49     assert(1 <= root && root <= n);
50     euler.clear();
51     depth[root] = 0;
52     root_dist[root] = 0;
53     dfs(root, 0);
54     int m = euler.size();
55     lg.assign(m + 1, 0);
56     for (int i = 2; i <= m; i++) lg[i] = lg[i / 2] + 1;
57     st.assign(lg[m] + 1, {});
58     st[0] = euler;
59     for (int k = 1; k <= lg[m]; k++)
60     {
61         st[k].resize(m - (1 << k) + 1);
62         for (int i = 0; i + (1 << k) <= m; i++)
63             st[k][i] = shallower(st[k - 1][i], st[k - 1][i + (1 << (k - 1))]);
64     }
65     ready = true;
66 }

```

LCA、边数距离与带权距离（接上页同一结构体）：

```
68     int lca(int u, int v) const
69     {
70         assert(ready && 1 <= u && u <= n && 1 <= v && v <= n);
71         int l = first[u], r = first[v];
72         if (l > r) swap(l, r);
73         int k = lg[r - l + 1];
74         return shallower(st[k][l], st[k][r - (1 << k) + 1]);
75     }
76
77     int distance(int u, int v) const
78     {
79         int z = lca(u, v);
80         return depth[u] + depth[v] - 2 * depth[z];
81     }
82
83     ll weighted_distance(int u, int v) const
84     {
85         int z = lca(u, v);
86         return (root_dist[u] - root_dist[z]) + (root_dist[v] - root_dist[z]);
87     }
88 };
```

第四章 树上点集摘要

4.1 树上点集直径摘要

默认空点集, $a=b=-1$ 、 $length=0$ 。insert($v,distance$) 插入顶点, 重复插入合法; merge($other,distance$) 合并两个点集摘要, 允许自合并; farthest($v,distance$) 返回 optional<pair<int,long long>> (最远顶点、距离), 空集返回 nullopt, 距离并列时可返回任意最远点。distance(u,v) 回调必须精确返回同一棵固定非负权树的距离, 点号为非负整数; 所有距离 $\leq LLONG_MAX$ 。每次插入或最远查询最多两次距离调用, 合并最多四次, 摘要只占 $O(1)$ 空间。保存 a 、 b 为点集中某对直径端点, 不保留全部点; 字段只读, 不能用一般图度量或负权树代替。复制摘要即可建立前缀、后缀或线段树聚合, 重置用默认构造。

```
1 struct TreeDiameter
2 {
3     using ll = long long;
4     int a = -1, b = -1;
5     ll length = 0;
6
7     // distance is the metric of one fixed tree with nonnegative edge weights.
8     template <class Distance> void insert(int v, const Distance &distance)
9     {
10         assert(v >= 0);
11         if (a == -1)
12         {
13             a = v;
14             b = v;
15             return;
16         }
17         ll x = distance(a, v), y = distance(b, v);
18         if (x >= y && x > length)
19         {
20             b = v;
21             length = x;
22         }
23         else if (y > length)
24         {
25             a = v;
26             length = y;
27         }
28     }
```

合并与最远点查询（接上页同一结构体）：

```
30     template <class Distance> void merge(TreeDiameter other, const Distance &distance)
31     {
32         if (other.a == -1) return;
33         insert(other.a, distance);
34         insert(other.b, distance);
35     }
36
37     template <class Distance>
38     optional<pair<int, ll>> farthest(int v, const Distance &distance) const
39     {
40         if (a == -1) return nullopt;
41         ll x = distance(a, v), y = distance(b, v);
42         return x >= y ? make_pair(a, x) : make_pair(b, y);
43     }
44 };
```

第五章 倍增祖先与路径最值

5.1 倍增 LCA、祖先与最大边权

构造 `LiftingLCA(n)`, `add(u,v,w=1)` 加边, `build(root=1)` 预处理非空无向树, 点号 `1..n`。 `jump(u,k)` 返回 `k` 级祖先, `k` 非负, 超过根返回 `0`; `lca(u,v)` 求最近公共祖先; `distance` 返回路径边数, 不是权值和; `max_edge` 返回路径最大边权, 同点返回 `nullopt`, 其他情况返回有值的 `optional<long long>`。支持负权和 `LLONG_MIN`, `LOW` 只是最大值合并的单位元, 不表示空路径。DFS 递归, 根深度为 `0`; `up[k][u]` 与 `mx[k][u]` 是 `2` 的 `k` 次方长度向上段的祖先与最大边权, `0` 号节点为哨兵。可重复 `build` 换根; 修改图后必须重建。预处理 $O(n \log n)$ 时间与空间, 各查询 $O(\log n)$ 。不需要边权最值时, 可删去 `mx` 和 `max_edge` 相关部分。全小树点对/根、极值权重及 `50` 万点负权链本地验证; P3379、POJ1330 驱动只验证 LCA, 最大边权仍是本地证据。

```
1 struct LiftingLCA
2 {
3     using ll = long long;
4     static constexpr ll LOW = numeric_limits<ll>::lowest();
5     int n, levels;
6     bool ready = false;
7     vector<vector<pair<int, ll>>> g;
8     vector<vector<int>> up;
9     vector<vector<ll>> mx;
10    vector<int> depth;
11
12    LiftingLCA(int n) : n(n), g(n + 1), depth(n + 1)
13    {
14        assert(n > 0);
15        levels = 1;
16        while ((1LL << levels) <= n) levels++;
17        up.assign(levels, vector<int>(n + 1));
18        mx.assign(levels, vector<ll>(n + 1, LOW));
19    }
20
21    void add(int u, int v, ll w = 1)
22    {
23        assert(1 <= u && u <= n && 1 <= v && v <= n);
24        g[u].push_back({v, w});
25        g[v].push_back({u, w});
26        ready = false;
27    }
```

```

29 void dfs(int u, int parent, ll weight)
30 {
31     up[0][u] = parent;
32     mx[0][u] = weight;
33     for (int k = 1; k < levels; k++)
34     {
35         int p = up[k - 1][u];
36         up[k][u] = up[k - 1][p];
37         mx[k][u] = max(mx[k - 1][u], mx[k - 1][p]);
38     }
39     for (auto [v, w] : g[u])
40         if (v != parent)
41         {
42             depth[v] = depth[u] + 1;
43             dfs(v, u, w);
44         }
45 }
46
47 // Nonempty tree only. Node 0 means no ancestor.
48 void build(int root = 1)
49 {
50     assert(1 <= root && root <= n);
51     depth[root] = 0;
52     dfs(root, 0, LOW);
53     ready = true;
54 }
55
56 int jump(int u, long long steps) const
57 {
58     assert(ready && 1 <= u && u <= n && steps >= 0);
59     if (steps > depth[u]) return 0;
60     for (int k = 0; k < levels; k++)
61         if (steps >> k & 1) u = up[k][u];
62     return u;
63 }

```

```

65  int lca(int u, int v) const
66  {
67      assert(ready && 1 <= u && u <= n && 1 <= v && v <= n);
68      if (depth[u] < depth[v]) swap(u, v);
69      u = jump(u, depth[u] - depth[v]);
70      if (u == v) return u;
71      for (int k = levels - 1; k >= 0; k--)
72          if (up[k][u] != up[k][v])
73              {
74                  u = up[k][u];
75                  v = up[k][v];
76              }
77      return up[0][u];
78  }
79
80  int distance(int u, int v) const
81  {
82      return depth[u] + depth[v] - 2 * depth[lca(u, v)];
83  }
84
85  optional<ll> max_edge(int u, int v) const
86  {
87      assert(ready && 1 <= u && u <= n && 1 <= v && v <= n);
88      if (u == v) return nullopt;
89      if (depth[u] < depth[v]) swap(u, v);
90      ll answer = LOW;
91      int delta = depth[u] - depth[v];
92      for (int k = 0; k < levels; k++)
93          if (delta >> k & 1)
94              {
95                  answer = max(answer, mx[k][u]);
96                  u = up[k][u];
97              }
98      if (u == v) return answer;
99      for (int k = levels - 1; k >= 0; k--)
100          if (up[k][u] != up[k][v])
101              {
102                  answer = max(answer, max(mx[k][u], mx[k][v]));
103                  u = up[k][u];
104                  v = up[k][v];
105              }
106      return max(answer, max(mx[0][u], mx[0][v]));
107  }
108 };

```

第六章 树上路径交

6.1 两条树路径的交集

`path_intersection(a,b,c,d,depth,lca)` 返回路径 `a-b` 与 `c-d` 的公共路径，字段 `u`、`v` 是交集端点，`vertices` 是公共顶点数；空交返回 `{0,0,0}`，单点交的 `u=v`。点号从 1 开始，`depth` 数组与 `lca` 回调必须来自同一棵非空树、同一个根，深度按边数计算，不能带带权根距离。支持数组或 `vector` 深度表，回调接收两个点并返回 `LCA`。将 `c`、`d` 分别投影到 `a-b` 得 `p`、`q`；投影是 `lca(a,b)`、`lca(a,x)`、`lca(b,x)` 中最深者。若 `p≠q`，`c-d` 必经过 `p-q`；若 `p=q`，检验 `p` 是否在 `c-d` 上即可区分单点交与空交。不依赖顶点编号大小。常数次 `LCA` 调用、`O(1)` 额外空间；使用 `EulerLCA` 时为 `O(1)`，`HLD/倍增` 时为 `O(log n)`。公共边数为 `max(0,vertices-1)`。全小树根与四端点、随机换号树用路径点集独立验证；P3398 完整程序只检查是否相交，端点及数量由本地验证覆盖。

```
1 struct PathIntersection
2 {
3     int u, v, vertices;
4 };
5
6 // Same nonempty tree and root for depth[] and lca(). Empty result is {0,0,0}.
7 template <class Depth, class Lca>
8 PathIntersection
9 path_intersection(int a, int b, int c, int d, const Depth &depth, Lca lca)
10 {
11     int z = lca(a, b);
12     auto project = [&](int x)
13     {
14         int p = z;
15         for (int v : {lca(a, x), lca(b, x)})
16             if (depth[v] > depth[p]) p = v;
17         return p;
18     };
19     auto distance = [&](int u, int v)
20     {
21         return 1LL * depth[u] + depth[v] - 2LL * depth[lca(u, v)];
22     };
23     int p = project(c), q = project(d);
24     if (p == q && distance(c, p) + distance(p, d) != distance(c, d)) return {0, 0, 0};
25     return {p, q, int(distance(p, q) + 1)};
26 }
```

第七章 点分治

7.1 点分治：树上距离点对计数

CentroidPairs(n), 点号 $1..n$; add(u,v,w) 建树, 允许森林但禁止环、重边、自环, 边权非负; 所有简单路径长度须能用 long long 表示。build() 预处理, 可重复调用; 改变边后须重建。count_leq(k) 统计距离 $\leq k$ 的无序不同顶点对, count_exact(k) 统计距离恰为 k 的点对; 负 k 返回 0, 零距离只统计不同顶点, 跨连通块不配对。parent[c] 是点分树父节点, 根为 0。all[c] 为该层分治块到重心的距离, 含重心距离 0; part[c] 为去掉重心后各子块的距离。预处理 $O(n \log^2 n)$, 空间 $O(n \log n)$, 每次计数 $O(n \log n)$; 固定树上多次查询可以复用排序结果。DFS 递归, 距离和比较采用减法避免溢出。不是动态点分树更新模板。

```
1 struct CentroidPairs
2 {
3     using ll = long long;
4     int n;
5     vector<vector<pair<int, ll>>> g;
6     vector<int> siz, removed, parent;
7     vector<vector<ll>> all;
8     vector<vector<vector<ll>>> part;
9
10    CentroidPairs(int n)
11        : n(n),
12          g(n + 1),
13          siz(n + 1),
14          removed(n + 1),
15          parent(n + 1),
16          all(n + 1),
17          part(n + 1)
18    {
19    }
20
21    void add(int u, int v, ll w)
22    {
23        assert(1 <= u && u <= n && 1 <= v && v <= n && u != v && w >= 0);
24        g[u].push_back({v, w});
25        g[v].push_back({u, w});
26    }
27
28    void size_dfs(int u, int p)
29    {
30        siz[u] = 1;
31        for (auto [v, w] : g[u])
32        {
33            if (v == p || removed[v]) continue;
34            size_dfs(v, u);
35            siz[u] += siz[v];
36        }
37    }
38
39    int centroid(int u, int p, int total)
40    {
41        for (auto [v, w] : g[u])
```

```

42     {
43         if (v != p && !removed[v] && siz[v] > total / 2)
44             return centroid(v, u, total);
45     }
46     return u;
47 }
48
49 void collect(int u, int p, ll d, vector<ll> &out)
50 {
51     out.push_back(d);
52     for (auto [v, w] : g[u])
53     {
54         if (v == p || removed[v]) continue;
55         assert(d <= numeric_limits<ll>::max() - w);
56         collect(v, u, d + w, out);
57     }
58 }
59
60 void decompose(int entry, int p)
61 {
62     size_dfs(entry, 0);
63     int c = centroid(entry, 0, siz[entry]);
64     removed[c] = 1;
65     parent[c] = p;
66     all[c] = {0};
67     for (auto [v, w] : g[c])
68     {
69         if (removed[v]) continue;
70         part[c].push_back({});
71         auto &d = part[c].back();
72         collect(v, c, w, d);
73         sort(d.begin(), d.end());
74         all[c].insert(all[c].end(), d.begin(), d.end());
75     }
76     sort(all[c].begin(), all[c].end());
77     for (auto [v, w] : g[c])
78     {
79         if (!removed[v]) decompose(v, c);
80     }
81 }
82
83 // Input must be a forest. Rebuild after changing edges.
84 void build()
85 {
86     fill(removed.begin(), removed.end(), 0);
87     for (int u = 1; u <= n; u++)
88     {
89         all[u].clear();
90         part[u].clear();
91     }
92     for (int u = 1; u <= n; u++)
93     {
94         if (!removed[u]) decompose(u, 0);
95     }
96 }
97

```

```

98  ll pairs(const vector<ll> &d, ll k) const
99  {
100      int l = 0, r = (int)d.size() - 1;
101      ll answer = 0;
102      while (l < r)
103      {
104          if (d[l] <= k - d[r])
105          {
106              answer += r - l;
107              l++;
108          }
109          else
110              r--;
111      }
112      return answer;
113  }
114
115  // Unordered pairs of distinct connected vertices. Nonnegative edge weights.
116  ll count_leq(ll k) const
117  {
118      if (k < 0) return 0;
119      ll answer = 0;
120      for (int c = 1; c <= n; c++)
121      {
122          answer += pairs(all[c], k);
123          for (const auto &d : part[c]) answer -= pairs(d, k);
124      }
125      return answer;
126  }
127
128  ll count_exact(ll k) const
129  {
130      if (k < 0) return 0;
131      return count_leq(k) - (k ? count_leq(k - 1) : 0);
132  }
133  };

```

第八章 树上启发式合并

8.1 树上启发式合并：子树众数颜色和

SubtreeColors(n), 点号 1..n, add(u,v) 建无向树; 要求图连通且无环, n=0 可空运行。run(colors,root=1) 传入长度 n+1 的 int 颜色数组, colors[0] 忽略。answer[u] 为子树内所有最高频颜色的原始编号之和, max_freq[u] 为该频次; 支持负颜色与稀疏编号, 内部离散化, 和用 long long。重复 run 可更换颜色或根。重儿子信息保留, 轻儿子通过 DFS 序区间加入; 清空整棵临时子树后再重置最大值, 不尝试逐次删除时维护众数。O(n log n) 时间, O(n) 空间, 全部 DFS 递归。insert 是内部增量统计, 可按题目修改统计量, 但相应清空规则也必须调整。

```
1 struct SubtreeColors
2 {
3     int n, best = 0;
4     long long sum = 0;
5     vector<vector<int>> g;
6     vector<int> siz, heavy, tin, tout, order, color, value, freq;
7     vector<int> max_freq;
8     vector<long long> answer;
9
10    SubtreeColors(int n) : n(n), g(n + 1) {}
11
12    void add(int u, int v)
13    {
14        assert(1 <= u && u <= n && 1 <= v && v <= n && u != v);
15        g[u].push_back(v);
16        g[v].push_back(u);
17    }
18
19    void prepare(int u, int p)
20    {
21        siz[u] = 1;
22        tin[u] = order.size();
23        order.push_back(u);
24        for (int v : g[u])
25        {
26            if (v == p) continue;
27            prepare(v, u);
28            siz[u] += siz[v];
29            if (siz[v] > siz[heavy[u]]) heavy[u] = v;
30        }
31        tout[u] = (int)order.size() - 1;
32    }
33
34    void insert(int u)
35    {
36        int c = color[u];
37        freq[c]++;
38        if (freq[c] > best)
39        {
40            best = freq[c];
41            sum = value[c];
42        }
```

```

43     else if (freq[c] == best)
44         sum += value[c];
45 }
46
47 void solve(int u, int p, bool keep)
48 {
49     for (int v : g[u])
50     {
51         if (v != p && v != heavy[u]) solve(v, u, false);
52     }
53     if (heavy[u]) solve(heavy[u], u, true);
54     for (int v : g[u])
55     {
56         if (v == p || v == heavy[u]) continue;
57         for (int i = tin[v]; i <= tout[v]; i++) insert(order[i]);
58     }
59     insert(u);
60     max_freq[u] = best;
61     answer[u] = sum;
62     if (!keep)
63     {
64         for (int i = tin[u]; i <= tout[u]; i++) freq[color[order[i]]]--;
65         best = 0;
66         sum = 0;
67     }
68 }
69
70 // A connected tree, colors[1..n]. Sum all colors tied for maximum frequency.
71 void run(const vector<int> &colors, int root = 1)
72 {
73     assert((int)colors.size() == n + 1);
74     siz.assign(n + 1, 0);
75     heavy = siz;
76     tin = siz;
77     tout = siz;
78     color = siz;
79     max_freq = siz;
80     answer.assign(n + 1, 0);
81     order.clear();
82     value.assign(colors.begin() + 1, colors.end());
83     sort(value.begin(), value.end());
84     value.erase(unique(value.begin(), value.end()), value.end());
85     freq.assign(value.size(), 0);
86     best = 0;
87     sum = 0;
88     if (!n) return;
89     assert(1 <= root && root <= n);
90     for (int u = 1; u <= n; u++)
91         color[u] =
92             lower_bound(value.begin(), value.end(), colors[u]) - value.begin();
93     prepare(root, 0);
94     solve(root, 0, false);
95 }
96 };

```

第九章 虚树

9.1 虚树：关键点压缩与路径信息

VirtualTree(n), $n \geq 1$, 点号 $1..n$; add($u,v,w=1$) 建非负权连通树, 无环和重边。prepare($root=1$) 递归预处理, 可换根重建; 根到任一点距离须在 long long 范围内。build(keys) 自动去重并补齐必要 LCA, 返回 vertices 和 edges; 全部编号沿用原树, vertices 按 DFS 序排列, 非空时首项即虚树根。edges 为 (from,to,distance,minimum), 分别是祖先到后代的虚边、压缩路径长度和路径最小边权。空关键点返回空结果, 单点返回单节点; 原树根不会强制加入, 需要时由调用者加入 keys。边按后代 DFS 序排列, 可逆序做自底向上的 DP。不分配或清空 n 个查询邻接表; 每次仅返回 $O(k)$ 结果。预处理 $O(n \log n)$ 时间空间, 单次 $O(k \log k + k \log n)$ 。lca、ancestor 也可直接使用, 要求有效点号且已经 prepare。构造用的祖先链栈属于虚树算法, 原树 DFS 保持递归。

```
1 struct VirtualTree
2 {
3     using ll = long long;
4
5     struct Edge
6     {
7         int from, to;
8         ll distance, minimum;
9     };
10
11    struct Result
12    {
13        vector<int> vertices;
14        vector<Edge> edges;
15    };
16
17    int n, log = 1, timer = 0;
18    vector<vector<pair<int, ll>>> g;
19    vector<vector<int>> up;
20    vector<vector<ll>> mn;
21    vector<int> depth, tin, tout;
22    vector<ll> dist;
23
24    VirtualTree(int n) : n(n), g(n + 1) {}
25
26    void add(int u, int v, ll w = 1)
27    {
28        assert(1 <= u && u <= n && 1 <= v && v <= n && u != v && w >= 0);
29        g[u].push_back({v, w});
30        g[v].push_back({u, w});
31    }
32
33    void dfs(int u, int p)
34    {
35        tin[u] = ++timer;
36        up[0][u] = p;
37        for (int j = 1; j < log; j++)
38        {
39            up[j][u] = up[j - 1][up[j - 1][u]];
```

```

40     mn[j][u] = min(mn[j - 1][u], mn[j - 1][up[j - 1][u]]);
41 }
42 for (auto [v, w] : g[u])
43 {
44     if (v == p) continue;
45     assert(dist[u] <= numeric_limits<ll>::max() - w);
46     depth[v] = depth[u] + 1;
47     dist[v] = dist[u] + w;
48     mn[0][v] = w;
49     dfs(v, u);
50 }
51 tout[u] = timer;
52 }
53
54 void prepare(int root = 1)
55 {
56     assert(1 <= root && root <= n);
57     log = 1;
58     while ((1LL << log) <= n) log++;
59     up.assign(log, vector<int>(n + 1));
60     mn.assign(log, vector<ll>(n + 1, numeric_limits<ll>::max()));
61     depth.assign(n + 1, 0);
62     tin = depth;
63     tout = depth;
64     dist.assign(n + 1, 0);
65     timer = 0;
66     dfs(root, root);
67 }
68
69 bool ancestor(int u, int v) const { return tin[u] <= tin[v] && tout[v] <= tout[u]; }
70
71 int lca(int u, int v) const
72 {
73     if (ancestor(u, v)) return u;
74     if (ancestor(v, u)) return v;
75     for (int j = log - 1; j >= 0; j--)
76     {
77         if (!ancestor(up[j][u], v)) u = up[j][u];
78     }
79     return up[0][u];
80 }
81
82 Edge path(int u, int v) const
83 {
84     assert(u != v && ancestor(u, v));
85     ll minimum = numeric_limits<ll>::max();
86     int x = v, steps = depth[v] - depth[u];
87     for (int j = 0; j < log; j++)
88     {
89         if (steps >> j & 1)
90         {
91             minimum = min(minimum, mn[j][x]);
92             x = up[j][x];
93         }
94     }
95     return {u, v, dist[v] - dist[u], minimum};

```

```

96     }
97
98     // Uses original vertex IDs. Empty input gives an empty result.
99     Result build(vector<int> keys) const
100     {
101         if (keys.empty()) return {};
102         for (int u : keys) assert(1 <= u && u <= n);
103         auto cmp = [&](int u, int v)
104         {
105             return tin[u] < tin[v];
106         };
107         sort(keys.begin(), keys.end(), cmp);
108         keys.erase(unique(keys.begin(), keys.end()), keys.end());
109         int k = keys.size();
110         for (int i = 1; i < k; i++) keys.push_back(lca(keys[i - 1], keys[i]));
111         sort(keys.begin(), keys.end(), cmp);
112         keys.erase(unique(keys.begin(), keys.end()), keys.end());
113         Result result{keys, {}};
114         vector<int> st;
115         for (int u : keys)
116         {
117             while (!st.empty() && !ancestor(st.back(), u)) st.pop_back();
118             if (!st.empty()) result.edges.push_back(path(st.back(), u));
119             st.push_back(u);
120         }
121         return result;
122     }
123 };

```

第十章 树上路径第 k 小

10.1 树上路径点权第 k 小

TreePathKth(a) 传入非空 0-based 权值数组，节点编号 1..n；add(u,v) 加无向边，要求恰好构成一棵树。build(root=1) 递归 DFS 建立根到各节点的频数版本与倍增表，可换根重建，旧版本自动清空。kth(u,v,k) 返回简单路径上第 k 小的点权，包含两个端点，每个节点计一次； $1 \leq k \leq$ 路径点数。lca(u,v) 返回当前预处理根下的最近公共祖先。使用 $root[u] + root[v] - root[lca] - root[parent(lca)]$ ，编号 0 为全零版本，避免少算或重复算 LCA。支持重复值与完整 long long 权值，内部只统计频数；权值和树结构在查询阶段保持不变。预处理 $O(n \log n)$ ，每次查询 $O(\log n)$ ，空间 $O(n \log n)$ 。递归深度最坏 $O(n)$ ，保持递归写法。

```
1 struct TreePathKth
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         int l = 0, r = 0, sum = 0;
8     };
9
10    int n, lg;
11    vector<vector<int>> g, up;
12    vector<int> root, dep;
13    vector<ll> value, vals;
14    vector<Node> t;
15
16    TreePathKth(const vector<ll> &a)
17        : n(a.size()), g(n + 1), root(n + 1), dep(n + 1), value(a)
18    {
19        assert(n > 0);
20        lg = 1;
21        while ((1LL << lg) <= n) lg++;
22        up.assign(lg, vector<int>(n + 1));
23    }
24
25    void add(int u, int v)
26    {
27        g[u].push_back(v);
28        g[v].push_back(u);
29    }
30
31    int insert(int old, int l, int r, int x)
32    {
33        int p = t.size();
34        Node copy = t[old];
35        t.push_back(copy);
36        t[p].sum++;
37        if (l == r) return p;
38        int m = (l + r) / 2;
39        if (x <= m)
40        {
41            int child = insert(t[old].l, l, m, x);
```

```

42         t[p].l = child;
43     }
44     else
45     {
46         int child = insert(t[old].r, m + 1, r, x);
47         t[p].r = child;
48     }
49     return p;
50 }
51
52 void dfs(int u, int p)
53 {
54     up[0][u] = p;
55     for (int j = 1; j < lg; j++) up[j][u] = up[j - 1][up[j - 1][u]];
56     int x = lower_bound(vals.begin(), vals.end(), value[u - 1]) - vals.begin();
57     root[u] = insert(root[p], 0, (int)vals.size() - 1, x);
58     for (int v : g[u])
59     {
60         if (v == p) continue;
61         dep[v] = dep[u] + 1;
62         dfs(v, u);
63     }
64 }
65
66 void build(int r = 1)
67 {
68     assert(1 <= r && r <= n);
69     vals = value;
70     sort(vals.begin(), vals.end());
71     vals.erase(unique(vals.begin(), vals.end()), vals.end());
72     t.assign(1, Node{});
73     dep[r] = 0;
74     dfs(r, 0);
75 }
76
77 int lca(int u, int v) const
78 {
79     if (dep[u] < dep[v]) swap(u, v);
80     int diff = dep[u] - dep[v];
81     for (int j = 0; j < lg; j++)
82         if (diff >> j & 1) u = up[j][u];
83     if (u == v) return u;
84     for (int j = lg - 1; j >= 0; j--)
85         if (up[j][u] != up[j][v])
86         {
87             u = up[j][u];
88             v = up[j][v];
89         }
90     return up[0][u];
91 }
92
93 ll kth(int u, int v, int k) const
94 {
95     int z = lca(u, v);
96     assert(1 <= k && k <= dep[u] + dep[v] - 2 * dep[z] + 1);
97     int a = root[u], b = root[v], c = root[z], d = root[up[0][z]];

```

```
98     int l = 0, r = (int)vals.size() - 1;
99     while (l < r)
100     {
101         int count = t[t[a].l].sum + t[t[b].l].sum - t[t[c].l].sum - t[t[d].l].sum;
102         int m = (l + r) / 2;
103         if (k <= count)
104         {
105             a = t[a].l;
106             b = t[b].l;
107             c = t[c].l;
108             d = t[d].l;
109             r = m;
110         }
111         else
112         {
113             k -= count;
114             a = t[a].r;
115             b = t[b].r;
116             c = t[c].r;
117             d = t[d].r;
118             l = m + 1;
119         }
120     }
121     return vals[l];
122 }
123 };
```

第十一章 动态树

11.1 Link-cut tree: 动态森林与路径异或

构造 LinkCutTree(n), 点号 1..n, 初始权值为零。set(x,value) 修改点权; link(x,y) 仅在不同连通块之间连边并返回是否成功; cut(x,y) 只删除真实存在的边, 否则返回 false。connected 查询连通性; query(x,y) 返回路径点权异或, 要求两点连通, 支持 x=y。make_root/find_root 维护与查询当前表示树根。每次操作均摊 $O(\log n)$, $O(n)$ 空间, splay 下传使用显式栈, 支持长链。权值为 unsigned long long。仅维护路径异或, 不包含虚子树聚合或动态直径。

```
1 struct LinkCutTree
2 {
3     using U = unsigned long long;
4
5     struct Node
6     {
7         int ch[2] {}, fa = 0;
8         U val = 0, sum = 0;
9         bool rev = false;
10    };
11
12    vector<Node> a;
13    vector<int> st;
14
15    LinkCutTree(int n) : a(n + 1), st(n + 1) {}
16
17    bool is_root(int x) const
18    {
19        int f = a[x].fa;
20        return a[f].ch[0] != x && a[f].ch[1] != x;
21    }
22
23    void pull(int x) { a[x].sum = a[a[x].ch[0]].sum ^ a[x].val ^ a[a[x].ch[1]].sum; }
24
25    void reverse(int x)
26    {
27        if (!x) return;
28        swap(a[x].ch[0], a[x].ch[1]);
29        a[x].rev = !a[x].rev;
30    }
31
32    void push(int x)
33    {
34        if (!a[x].rev) return;
35        reverse(a[x].ch[0]);
36        reverse(a[x].ch[1]);
37        a[x].rev = false;
38    }
39
40    void rotate(int x)
41    {
42        int y = a[x].fa, z = a[y].fa;
```

```

43     int k = a[y].ch[1] == x;
44     int w = a[x].ch[k ^ 1];
45     if (!is_root(y)) a[z].ch[a[z].ch[1] == y] = x;
46     a[x].fa = z;
47     a[y].ch[k] = w;
48     if (w) a[w].fa = y;
49     a[x].ch[k ^ 1] = y;
50     a[y].fa = x;
51     pull(y);
52     pull(x);
53 }
54
55 void splay(int x)
56 {
57     int top = 0, y = x;
58     st[top++] = y;
59     while (!is_root(y))
60     {
61         y = a[y].fa;
62         st[top++] = y;
63     }
64     while (top) push(st[--top]);
65     while (!is_root(x))
66     {
67         int y = a[x].fa, z = a[y].fa;
68         if (!is_root(y))
69         {
70             bool same = (a[y].ch[1] == x) == (a[z].ch[1] == y);
71             rotate(same ? y : x);
72         }
73         rotate(x);
74     }
75 }
76
77 void access(int x)
78 {
79     int last = 0;
80     for (int y = x; y; y = a[y].fa)
81     {
82         splay(y);
83         // The old right child's fa remains y: it becomes a path-parent.
84         a[y].ch[1] = last;
85         if (last) a[last].fa = y;
86         pull(y);
87         last = y;
88     }
89     splay(x);
90 }
91
92 void make_root(int x)
93 {
94     access(x);
95     reverse(x);
96 }
97
98 int find_root(int x)

```

```

99     {
100         access(x);
101         push(x);
102         while (a[x].ch[0])
103         {
104             x = a[x].ch[0];
105             push(x);
106         }
107         splay(x);
108         return x;
109     }

110
111     bool connected(int x, int y) { return x == y || find_root(x) == find_root(y); }
112
113     bool link(int x, int y)
114     {
115         make_root(x);
116         if (find_root(y) == x) return false;
117         a[x].fa = y;
118         return true;
119     }
120
121     bool cut(int x, int y)
122     {
123         make_root(x);
124         access(y);
125         if (a[y].ch[0] != x || a[x].ch[1]) return false;
126         a[y].ch[0] = 0;
127         a[x].fa = 0;
128         pull(y);
129         return true;
130     }
131
132     void set(int x, U value)
133     {
134         access(x);
135         a[x].val = value;
136         pull(x);
137     }
138
139     // x and y must be connected. This is a path aggregate, not a subtree sum.
140     U query(int x, int y)
141     {
142         make_root(x);
143         access(y);
144         return a[y].sum;
145     }
146 };

```

第十二章 配套知识与验证范围

12.1 非负权树的点集直径摘要

代码见第 8 页。一个非空点集只需保存任意一对直径端点 a, b ，即可回答任意树上点 x 到该点集的最远距离： $\max(d(x, a), d(x, b))$ 。边权非负，包括零权；该结论不能推广到一般图或负权树。

证明可将点 u 投影到路径 $a-b$ ，记投影距 a 为 t ，离路径的距离为 h ，直径长为 D 。由于 $d(a, u), d(b, u) \leq D$ ，有 $h \leq \min(t, D - t)$ 。再将查询点投影到同一路径，投影距 a 为 s ，离路径的距离为 g 。经两个投影点连接得到 $d(x, u) \leq g + h + |s - t| \leq g + \max(s, D - s)$ ，右侧恰为 $\max(d(x, a), d(x, b))$ 。因此插入新点只需比较到两个旧端点的距离；合并点集时，只插入另一个摘要的两个端点即可。并列直径可任选，不应假设端点编号固定或合并后的端点与合并顺序无关。

Euler LCA 现在同时保存根距离，但其原有 `distance` 接口仍返回边数。带权距离请显式调用 `weighted_distance / Weighted_Distance`；采用 $(D_u - D_z) + (D_v - D_z)$ 避免不必要的两倍根距离中间值。所有路径权和须能用 `long long` 表示。独立验证穷举小点集及合并，逐点检查最远距离与返回顶点；带权路径用独立树遍历参照，并覆盖全部根、零边权与 `LLONG_MAX` 单边。原有 Euler LCA 全小树回归，以及 50 万点、边权 10^9 的递归链也通过普通与 ASan/UBSan 检查。

杭州 2023 F 中，点权互异。令 g 为全树点权的 `mex`，只需按点权 $0, 1, \dots, g - 1$ 建立前缀摘要。某个前缀全部位于以 x 为中心、半径 k 的范围内，当且仅当该摘要的最远距离不超过 k 。二分最长满足条件的前缀长度即为答案； $g = 0$ 时直接为零。题面树边带权，不能用边数距离。完整驱动在 `verify/qoj/8235.*.cpp`，本地使用实际权距与 `mex` 集合参照， $n = q = 500000$ 的链和星形树保持递归 DFS。两套驱动各 50 个正式测试和 1 个额外测试全部在线 AC：动态版 666 ms、198076 KB，传统版 588 ms、198196 KB。OJ 验证题面正权边、前缀插入和最远距离判定；零权边、完整 64 位距离与任意摘要合并仍由独立本地验证支持。

算法索引

CentroidPairs, 14

EulerLCA, 5

HLD, 1

LiftingLCA, 10

LinkCutTree, 25

OfflineLCA, 3

path_intersection, 13

SubtreeColors, 17

TreeDiameter, 8

TreePathKth, 22

VirtualTree, 19